Unit IV

4. Tree:

4.1 Introduction
4.2 Terminology of Binary tree
4.3 Types of Binary tree
4.4 Traversing of binary tree
4.5 Header Nodes, Threads
4.6 General Tree Introduction

4.1 Introduction to Trees:

Definition:

A **tree** is a hierarchical data structure consisting of **nodes** connected by **edges**. It is a non-linear structure where one node is designated as the root, and every other node is connected directly or indirectly to the root.

Key Terminology:

1. Node:

A basic unit of a tree containing data.

2. Root:

The topmost node in a tree, with no parent.

3. Edge:

A connection between two nodes.

4. Parent:

A node that has child nodes.

5. Child:

A node that is a descendant of another node.

6. Leaf Node:

A node with no children.

7. Subtree:

A portion of a tree that itself is a tree.

8. Height:

The number of edges on the longest path from the root to a leaf.

9. Depth:

The number of edges from the root to a given node.

10. Degree:

The number of children a node has.

Properties of a Tree:

1. A tree with n nodes has $n - 1$ edges.
2. There is exactly one path between any two nodes.
3. Trees are connected and acyclic (no cycles).

Types of Trees:

1. Binary Tree:

Each node has at most two children (left and right).

2. Binary Search Tree (BST):

A binary tree where the left child contains values smaller than the parent, and the right child contains values greater than the parent.

3. Full Binary Tree:

Every node has 0 or 2 children.

4. Complete Binary Tree:

All levels are completely filled except possibly the last level, which is filled from left to right.

5. Balanced Tree:

The height difference between the left and right subtrees of any node is minimal (e.g., AVL tree).

Representation of Trees in Memory:

1. Linked Representation:

Each node contains:

Data.
Pointer to the left child.
Pointer to the right child.

Example:

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

2. Array Representation:

Used for complete binary trees.

For a node at index i:

Left child is at $2i + 1$.
Right child is at $2i + 2$.
Parent is at $\lfloor (i - 1)/2 \rfloor$.

Applications of Trees:

1. **Hierarchical Data Storage:**
   File systems, XML/HTML data.

2. **Searching and Sorting:**
   Binary search trees, heaps.

3. **Routing Algorithms:**
   Spanning trees in networks.

4. **Expression Evaluation:**
   Expression trees for arithmetic operations.

Advantages of Trees:

1. Hierarchical structure simplifies complex relationships.
2. Efficient for searching, insertion, and deletion in binary search trees.

Disadvantages of Trees:

1. Complex implementation compared to linear structures.
2. Balancing can be computationally expensive for some tree types.

4.2 Terminology of Binary tree:

Definition:

A **binary tree** is a type of tree data structure in which each node has at most **two children**, referred to as the **left child** and **right child**.

Key Terminology of Binary Tree:

1. **Node:**
   A fundamental unit of the tree that contains data and links to its children.

2. **Root:**
   The topmost node in the binary tree.
   It has no parent.
   Example:

```
10
```

3. **Edge:**
   A connection between a parent node and a child node.
   Represents the relationship between nodes.

4. **Parent:**
   A node that has one or more children.
   Example:

```
10
/ \
5   20
```

Here, **10** is the parent of **5** and **20**.

5. Child:

A node that is a descendant of another node.

Example:

In the above tree, **5** and **20** are children of **10**.

6. Leaf Node:

A node that has no children (left and right pointers are NULL).

Example:

In the tree above, **5** and **20** are leaf nodes.

7. Internal Node:

A node that is not a leaf (has at least one child).

Example:

In the above tree, **10** is an internal node.

8. Level:

The depth of a node from the root, starting at level 0 for the root.

Example:

```
10  --> Level 0
/  \
5    20  --> Level 1
```

9. Height:

The number of edges on the longest path from a node to a leaf.

Height of a tree = Height of the root node.

10. Depth:

The number of edges from the root to a specific node.

11. Subtree:

A tree consisting of a node and its descendants.

Example:

In the above tree, the subtree rooted at **5** is:

5

12. Degree:

The number of children a node has.

In a binary tree, the degree of a node can be 0, 1, or 2.

13. Sibling:

Nodes that share the same parent.

Example:

In the above tree, **5** and **20** are siblings.

14. Binary Tree Types:

Full Binary Tree:

Every node has 0 or 2 children.

Complete Binary Tree:

All levels are fully filled except possibly the last, filled from left to right.

Perfect Binary Tree:

All internal nodes have 2 children, and all leaf nodes are at the same level.

Skewed Binary Tree:

All nodes have only one child (left-skewed or right-skewed).

Applications of Binary Trees:

1. Represent hierarchical data like file systems.
2. Efficient searching and sorting (binary search trees).
3. Expression evaluation in expression trees.

4.3 Types of Binary tree:

Definition:

A **binary tree** is a hierarchical data structure where each node has at most two children, referred to as the **left child** and **right child**. There are several specialized types of binary trees based on their structure and properties.

1. Full Binary Tree:

A binary tree in which every node has either **0 or 2 children**.

No node in a full binary tree has only one child.

Example:

```
1
/ \
2   3
/ \
4   5
```

Properties:

1. Total nodes = $2^h - 1$ (where h is the height of the tree).
2. All non-leaf nodes have exactly 2 children.

2. Complete Binary Tree:

A binary tree in which:

All levels, except possibly the last, are completely filled.

The last level has nodes filled from **left to right**.

Example:

```
      1
     / \
    2   3
   / \
  4   5
```

Properties:

1. All leaf nodes are as far left as possible.
2. Commonly used in heap implementations.

3. Perfect Binary Tree:

A binary tree in which all internal nodes have **2 children**, and all leaf nodes are at the **same level**.

Example:

```
      1
     / \
    2   3
   / \ / \
  4  5 6  7
```

Properties:

1. Total nodes = $2^h - 1$, where h is the height.
2. Number of leaf nodes = 2^{h-1}.

4. Skewed Binary Tree:

A binary tree where all nodes have only one child.

Can be:

Left-Skewed:

All children are on the left.

Right-Skewed:

All children are on the right.

Example (Left-Skewed):

```
      1
     /
    2
   /
  3
```

Properties:

1. Height of the tree = Number of nodes - 1.
2. Degrades to linear structure, similar to a linked list.

5. Balanced Binary Tree:

A binary tree in which the **height difference between the left and right subtrees of every node** is at most 1.

Example: AVL Tree

Properties:

1. Ensures logarithmic time complexity for search, insertion, and deletion.
2. Commonly used for efficient searching.

6. Binary Search Tree (BST):

A binary tree where:

The left subtree of a node contains only nodes with values **less than** the node's value.

The right subtree of a node contains only nodes with values **greater than** the node's value.

Example:

```
    10
    / \
   5   20
  / \
 2   8
```

Properties:

1. Provides efficient searching, insertion, and deletion $(O(\log n) in\ average\ cases)$.
2. In-order traversal yields a sorted sequence.

4.4 Traversing of binary tree:

Definition:

Tree traversal refers to the process of visiting all the nodes of a binary tree in a specific order. During traversal, each node is processed (e.g., its value is printed, accessed, or modified) exactly once.

Types of Tree Traversals:

Tree traversals are mainly categorized into **two types**:

1. Depth-First Traversal:

Depth-first traversal explores the depth of a binary tree by visiting nodes in one branch before moving to another. There are three subtypes:

a) Preorder Traversal (NLR):

Process the **Node**, then traverse the **Left subtree**, and finally traverse the **Right subtree**.

Algorithm:

1. Visit the root.
2. Traverse the left subtree in preorder.
3. Traverse the right subtree in preorder.

Example:

For the binary tree:

```
      1
     / \
    2   3
   / \
  4   5
```

Preorder: `1, 2, 4, 5, 3`

b) Inorder Traversal (LNR):

Traverse the **Left subtree**, process the **Node**, and then traverse the **Right subtree**.

Algorithm:

1. Traverse the left subtree in inorder.
2. Visit the root.
3. Traverse the right subtree in inorder.

Example:

For the binary tree:

```
      1
     / \
    2   3
   / \
  4   5
```

Inorder: `4, 2, 5, 1, 3`

c) Postorder Traversal (LRN):

Traverse the **Left subtree**, then the **Right subtree**, and finally process the **Node**.

Algorithm:

1. Traverse the left subtree in postorder.
2. Traverse the right subtree in postorder.
3. Visit the root.

Example:

For the binary tree:

```
      1
     / \
    2   3
   / \
```

4 5

Postorder: `4, 5, 2, 3, 1`

2. Breadth-First Traversal (Level Order Traversal):

In breadth-first traversal, nodes are visited level by level, starting from the root and moving to the next level.

Algorithm:

1. Start at the root node.
2. Visit all nodes at the current level.
3. Move to the next level and repeat.

Example:

For the binary tree:

```
      1
     / \
    2   3
   / \
  4   5
```

Level Order: `1, 2, 3, 4, 5`

Applications of Tree Traversals:

1. Preorder:

Used to create a copy of the tree or prefix expressions.

2. Inorder:

Used for binary search trees (BST) to retrieve data in sorted order.

3. Postorder:

Used for deleting a tree or evaluating postfix expressions.

4. Level Order:

Used in breadth-first search (BFS) algorithms.

4.5 Header Nodes, Threads:

1. Header Nodes:

Definition:

A **header node** is a special node used in certain data structures (like threaded binary trees or linked lists) to serve as a placeholder or marker. It often helps in simplifying operations like traversal, insertion, and deletion by providing a consistent entry point to the structure.

Usage in Binary Trees:

1. Acts as a starting point for traversal.

2. Contains pointers to the root of the tree and other metadata, such as tree size or height.
3. Can help manage special cases like empty trees.

Example:

In a threaded binary tree, the header node might point to the root of the tree and facilitate circular links.

2. Threads in Binary Trees:

Definition:

Threads are pointers added to a binary tree's nodes to make traversal faster and avoid the use of recursion or stacks. These pointers replace `NULL` links in leaf nodes and point to the **inorder predecessor** or **successor** of the node.

Threaded Binary Tree:

A **threaded binary tree** is a binary tree where threads are introduced to help traverse the tree in an efficient manner.

Types of Threaded Binary Trees:

1. Single Threaded:

A node contains a thread pointer to either its **inorder predecessor** or **inorder successor**.

2. Double Threaded:

A node contains two thread pointers: one for the **inorder predecessor** and one for the **inorder successor**.

Advantages of Threads:

1. Eliminates the need for recursion or a stack during traversal, saving memory.
2. Traversal becomes faster as threads directly connect nodes in the traversal order.

Example of Inorder Threading:

Consider this binary tree:

```
  1
 / \
2   3
```

Threaded Representation:

The left thread of `2` points to its **inorder predecessor** (if any).
The right thread of `3` points to its **inorder successor** (if any).

Applications of Threads:

1. Useful in tree traversal when memory usage needs to be minimized.
2. Efficient for in-order traversal of binary search trees.

4.6 General Tree Introduction:

Definition:

A **general tree** is a hierarchical data structure where each node can have an arbitrary number of children. Unlike binary trees, which limit nodes to two children, general trees allow flexibility in the number of descendants a node can have.

Key Characteristics of General Trees:

1. Node:

The basic element of the tree that holds data and references to its children.

2. Parent and Child Relationship:

Each node can have one parent and multiple children.

3. Root Node:

The topmost node with no parent.

4. Leaf Node:

A node with no children.

5. Internal Node:

A node with at least one child.

6. Height of the Tree:

The longest path from the root to a leaf.

7. Degree of a Node:

The number of children a node has.

Tree Representation in Memory:

General trees can be represented in memory using two common methods:

1. Parent-Child Representation:

Each node contains a pointer to its **first child** and a pointer to its **next sibling**.
This is known as the **first-child, next-sibling representation**.

Example:

For this general tree:

```
    A
   / | \
  B  C  D
```

Memory Representation:

A -> First Child: B
B -> Next Sibling: C
C -> Next Sibling: D
D -> No Sibling

2. Dynamic Lists:

Each node contains a dynamic list (or array) of pointers to its children.

Applications of General Trees:

1. **Hierarchical Data:**
   Organization charts, file systems, and XML/HTML documents.

2. **Game Trees:**
   Representing possible moves in a game like chess.

3. **Parsing Trees:**
   Used in compilers for parsing expressions or code.

Advantages of General Trees:

1. Flexibility in the number of child nodes.
2. Suitable for hierarchical and structured data.

Disadvantages of General Trees:

1. Traversal and manipulation can be more complex compared to binary trees.
2. Memory management requires dynamic structures for arbitrary child nodes.